

DRCS Engine — Testing Strategy

How the engine is tested, what's covered, and how to extend the test suite.

Test Philosophy

1. **Gates are pure functions** — inputs → outputs, no hidden state. Test them in isolation.
 2. **Orchestrator is integration** — test the full gate sequence with real persistence (in-memory DB).
 3. **LLM calls are always mocked** — zero network requests in CI. Fast, deterministic, repeatable.
 4. **Tenant isolation is non-negotiable** — every test suite includes multi-tenant verification.
 5. **Fail-closed guarantees are verified** — C3 must block on log integrity issues, never silently pass.
-

Test Suite Inventory

Gates (Unit Tests)

Suite	File	Tests	Coverage
C1 Lock	tests/c1.test.ts	4	Unsealed pass, sealed fail, tenant isolation, idempotent sealing
C2 Bank	tests/c2.test.ts	6	Explicit category_id, situation match, no match, protected categories, tenant isolation
C3 Governor	tests/c3.test.ts	7	Under cap (allow), at cap (block), rolling window, fail-closed, tenant isolation
C4 Resolution	tests/c4.test.ts	12	3-step ladder (as-is/recaption/escalate), caption normalization, exclude/required_tag filters, errors
C5 Generation	tests/c5.test.ts	8	Prompt construction, response parsing (clean JSON/fenced/plaintext), generation success/failure
C6 Governance	tests/c6.test.ts	7	All 4 dispositions reachable, no silent downgrade, descriptive fields, records, tenant isolation

Total: 44 gate tests

Infrastructure (Integration Tests)

Suite	File	Tests	Coverage
Orchestrator	tests/orchestrator.test.ts	30+	Full gate sequence, short-circuiting, evaluatePrompt LLM flow, verdict structure
Persistence	tests/persistence.test.ts	15	Prisma client, migrations, Zilly seed, tenant-scoped queries, idempotency
Assets	tests/assets.test.ts	10	resolveAsset, findAssetByCategory, file_path handling, tenant isolation
Media	tests/media.test.ts	10	(placeholder / media generator provider tests — verify mock generation flow)

Total: 65+ infrastructure tests

Overall Coverage

- **107 tests** across 10 test suites
 - **All passing** (verified 2026-08-05)
 - **Zero network calls** (all LLM calls mocked)
 - **In-memory SQLite** (`test.db`) reset before each suite
-

Testing Layers

Layer 1 — Pure Gate Logic (Unit)

What: Test individual gates in complete isolation.

How: Import gate function, call with test data, assert output matches contract.

Example:

```
import { selectCategory } from '../src/gates/c2';
import { categorySchema } from '../src/persistence';

test('C2 resolves exact category_id match', async () => {
  await categorySchema.upsertCategory('test-tenant', {
    category_id: 'victory',
    name: 'Victory',
    asset_list: ['asset_001'],
  });

  const result = await selectCategory(
    { category_id: 'victory' },
    'test-tenant'
  );

  expect(result.matched).toBe(true);
  expect(result.category_id).toBe('victory');
  expect(result.available_assets).toEqual(['asset_001']);
});
```

Why: Verifies gate contract in isolation. If this test fails, the gate logic is broken — no orchestrator or LLM involved.

Layer 2 — Persistence Integration (Integration)

What: Verify data flows correctly through Prisma ORM → SQLite/PostgreSQL.

How: Seed data, query it, assert tenant isolation and idempotency.

Example:

```
import { assetRegistry } from '../src/persistence';

test('AssetRegistry respects tenant isolation', async () => {
  await assetRegistry.createAsset('tenant-a', {
    asset_id: 'shared-id',
    tag: 'canonical',
    category: 'test',
  });

  const result = await assetRegistry.getAsset('tenant-b', 'shared-id');
  expect(result).toBeNull(); // tenant-b cannot see tenant-a's asset
});
```

Why: Confirms multi-tenant isolation at the persistence layer — the foundation of the entire architecture.

Layer 3 — Orchestrator Sequence (End-to-End)

What: Test the full gate pipeline (C6 → C2 → C4 → C5 → C3) with real persistence and mocked LLM.

How: Call `evaluate()` or `evaluatePrompt()`, assert verdict structure, check audit trail.

Example:

```
import { evaluatePrompt } from '../src/orchestrator';
import * as llm from '../src/llm';

test('evaluatePrompt returns PUBLISH verdict with file_path', async () => {
  // Mock LLM to return a known category
  jest.spyOn(llm, 'callLLM').mockResolvedValue(
    JSON.stringify({ category: 'Victory', content_type: 'clip' })
  );

  const verdict = await evaluatePrompt('zilly', 'hype post about 10k');

  expect(verdict.decision).toBe('PUBLISH');
  expect(verdict.file_path).toBe('media/assets/zilly_mascot.png');
  expect(verdict.caption).toBe('Victory');
  expect(verdict.trail).toHaveLength(5); // C6, C2, C4, C5, C3
});
```

Why: Proves the engine works end-to-end, from natural language input to media file output.

LLM Mocking Strategy

Why Mock:

- **Speed:** No network latency, tests run in ~5 seconds
- **Determinism:** Same input → same output, every run
- **Cost:** Zero API usage in CI
- **Offline:** Tests work without internet

How:

```
import * as llm from '../src/llm';

jest.spyOn(llm, 'callLLM').mockResolvedValue({'caption': 'test'});
```

What We Mock:

- C5 generation (buildPrompt → LLM → parseGeneration)
- evaluatePrompt field extraction (extractPromptFields → LLM)

What We Don't Mock:

- C5's buildPrompt function (tested directly — ensures correct LLM prompt structure)
- C5's parseGeneration function (tested with various LLM response shapes — clean JSON, fenced JSON, plaintext)

Manual LLM Integration Test (Run Locally):

```
ABACUS_API_KEY=<your-key> npm run evaluate -- --prompt "test idea"
```

This is the **one** place you verify the LLM integration actually works. CI tests verify everything except the real LLM call.

Test Database Strategy

Development:

- `prisma/dev.db` — persistent SQLite, manually seeded, used during local dev

Testing:

- `prisma/test.db` — in-memory SQLite, reset before each test suite
- Schema auto-migrated via `jest.config.js` global setup
- Zero cross-test contamination

Production:

- PostgreSQL (connection string in `DATABASE_URL` env var)
 - Tests run against PostgreSQL in staging before production deploy
-

Tenant Isolation Verification

Every test suite MUST include a tenant isolation test.

Pattern:

```
test('[Gate Name] respects tenant isolation', async () => {
  // Seed data for tenant-a
  await persistence.create('tenant-a', {...});

  // Query as tenant-b
  const result = await gate.function('tenant-b', ...);

  // Assert tenant-b sees nothing from tenant-a
  expect(result).toBeNull(); // or .matched = false, etc.
});
```

Why: Multi-tenant isolation is a hard constraint. If any gate leaks data cross-tenant, the entire architecture fails.

Acceptance Criteria Verification

Every gate has acceptance criteria defined in its design doc (`01-gates/*`). Tests verify **every** criterion.

Example — C3 (Governor):

Acceptance Criterion	Test
Allows deployment when count < max_count	C3 allows when under cap
Blocks when count >= max_count	C3 blocks when at cap
Rolling window excludes old deployments	C3 rolling window excludes deployments older than 30 days
Fail-closed: blocks on log integrity issue	C3 fails closed on usage log integrity issue
Tenant isolation	C3 respects tenant isolation

If any acceptance criterion is NOT tested, it is NOT verified.

Coverage Gaps (Intentional)

1. **Real LLM calls** — mocked in all automated tests. Manual verification required.
2. **PostgreSQL production DB** — tests run on SQLite. Staging environment tests PostgreSQL.
3. **Web console UI** — no Playwright/Cypress tests yet. Manual QA required.
4. **CLI edge cases** — basic `--prompt` flow tested, advanced flags (`--dry-run` , `--exclude`) not yet covered.
5. **C7-C8 gates** — don't exist yet, so no tests.

Adding New Tests

Adding a Test for a New Gate (Future C7/C8)

1. **Create test file:** `tests/c7.test.ts`
2. **Define acceptance criteria** in `01-gates/C7-*/c7.md`
3. **Write one test per criterion:**
 - Gate logic in isolation
 - Tenant isolation
 - Error handling
 - Edge cases
4. **Run:** `npm test -- tests/c7.test.ts`
5. **Verify:** All tests pass before merging

Adding a Test for Orchestrator Behavior

1. **Identify the scenario** (e.g., "C6 REJECT should short-circuit before C2")
2. **Seed test data** (governance policy that rejects)
3. **Call** `evaluate()` with the trigger

4. Assert:

- `verdict.decision === 'BLOCKED'`
- `verdict.stopped_at_gate === 'C6'`
- `verdict.trail` has only 1 entry (C6)

5. File: `tests/orchestrator.test.ts`

Adding a Persistence Test

1. Choose the repository (e.g., `governanceRecord`)**2. Test CRUD operations:**

- Create a record
- Read it back
- Update it (if applicable)
- List records (with filters)

3. Test tenant isolation:

- Create for tenant-a
- Query as tenant-b
- Assert null/empty

4. File: `tests/persistence.test.ts`

Continuous Integration (CI) Checklist

Before merging any PR:

- [] `npm test` passes (all 107 tests green)
- [] `npm run build` succeeds (TypeScript clean)
- [] No new ESLint warnings (if linter configured)
- [] Coverage doesn't drop (if tracking coverage %)
- [] Manual LLM integration test passes (one prompt → real verdict)

Test Maintenance Rules

- 1. Never commit a failing test.** If a test fails, fix it or delete it — don't leave it red.
- 2. Never skip tests.** Use `test.skip()` only temporarily during debugging, never in merged code.
- 3. Keep tests fast.** Target: full suite < 10 seconds. If it grows slower, profile and optimize.
- 4. Mock expensive operations.** LLM calls, external APIs, slow DB queries — all mocked.
- 5. One assertion per concept.** Don't test 5 things in one `it()` block — split into 5 tests.

Common Test Patterns

Pattern: Seeding Test Data

```
beforeEach(async () => {
  await categorySchema.upsertCategory('test-tenant', {
    category_id: 'test-cat',
    name: 'Test Category',
    asset_list: ['asset_001'],
  });
  await assetRegistry.createAsset('test-tenant', {
    asset_id: 'asset_001',
    tag: 'canonical',
    category: 'test-cat',
    caption: 'Test caption',
    file_path: 'test/path.png',
  });
});
```

Pattern: Mocking LLM

```
jest.spyOn(llm, 'callLLM').mockResolvedValue({'caption': 'Generated'});
```

Pattern: Asserting Audit Trail

```
expect(verdict.trail).toHaveLength(5); // C6, C2, C4, C5, C3
expect(verdict.trail[0].gate).toBe('C6');
expect(verdict.trail[0].passed).toBe(true);
```

Pattern: Testing Fail-Closed

```
// Corrupt the log by deleting entries
await prisma.usageLog.deleteMany({ where: { tenant_id: 'test-tenant' } });

const result = await checkRepetition('asset_001', { tenant_id: 'test-tenant' });

expect(result.allowed).toBe(false);
expect(result.failed_closed).toBe(true);
```

Performance Benchmarks

Metric	Target	Current
Full test suite runtime	< 10 seconds	~5 seconds
Single gate test	< 100ms	~20ms
Orchestrator end-to-end test	< 500ms	~200ms

Why Speed Matters: Fast tests encourage running them often. Slow tests get skipped.

Next Steps

1. Add **Playwright tests** for web console (prompt input → verdict display)
 2. Add **PostgreSQL integration tests** in staging environment
 3. Track **test coverage %** (aim for 90%+ on gate logic, 80%+ on orchestrator)
 4. Add **load testing** (how many `evaluatePrompt` calls/sec can the engine handle?)
 5. Add **fuzz testing** (random inputs to C2/C4/C5 to find edge-case crashes)
-

Remember: The test suite is the second source of truth (after the locked scripts). If it's not tested, it's not verified. If it's not verified, it's not shipped.